

Appendix: Measurement Methodology

Reproduction, evidence scope, and the limits of the measurements

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

Repository: github.com/matanco64/HWSW_Project

A1. Timing and provenance

The headline results use the course QEMU/KVM virtual machine (VM), Ubuntu 22.04, release CPython 3.10.12 and pyperformance 1.14.0 with pyperf 2.10.0. Each JSON contains 120 measured values (40 value-bearing worker runs, three values each); calibration is separate. Reported uncertainty is sample standard deviation across those values, not a confidence interval. Use the full-precision JSON means to calculate ratios, rather than rounded display values.

Comparison	Files in the results directory
Nbody original / Python	vm_canonical_20260910_2c8c754/suite/{baseline,optimized}_nbody.json
Nbody Python / native	vm_canonical_20260910_2c8c754/suite/{fallback,native}_nbody.json
Pyflate all tiers	vm_canonical_20260910_2c8c754/suite/{baseline,optimized,fallback,native}_pyflate.json

Original/optimized use pyperformance; fallback/native use direct pyperf on the optimized benchmark. These are different pairs, so their ratios must not be multiplied as one paired experiment. A7 maps backend, affinity and binary metadata to the canonical run. Development hosts and older captures are named in A2 and A7 below; the revision history of this appendix is in the repository's git log.

Reproducing the comparisons

From the repository root, compare the preserved data:

```
suite=results/vm_canonical_20260910_2c8c754/suite
python3 -m pyperf compare_to "$suite/baseline_nbody.json" \
"$suite/optimized_nbody.json"
python3 -m pyperf compare_to "$suite/fallback_pyflate.json" \
"$suite/native_pyflate.json"
```

The root benchmark scripts provide the complete workflow (README.md). For a fresh pinned native comparison on the prepared course VM:

```
./tools/build_wheel.sh nbody --record /tmp/nbody-wheel.json
HWSW_BACKEND=native taskset -c 0 \
python3 benchmarks/bm_nbody/run_benchmark.py \
--rigorous --inherit-environ HWSW_BACKEND -o /tmp/nbody-native.json
```

Repeat with HWSW_BACKEND=python and a fresh filename; use pyflate / bm_pyflate for decompression. build_wheel.sh installs the exact newly built wheel and records source, module and wheel hashes. Direct pyperf uses the interpreter containing that extension; pyperformance manages a separate environment. Native mode fails if the extension is missing. The runner validates worker backend requests, actual backend and CPU affinity in each JSON.

Correctness contracts

- **Nbody shipped Python and native:** compare all state components and energy against the original after identical initial conditions and 20,000 steps. dev/nbody/verify.py gates the landed kernel on exact equality; its development tiers use tolerances. dev/nbody/rs_check.py defaults to the exact contract.
- **Nbody hardware:** compare RTL with its own arithmetic model, then compare the original using the declared energy and position/velocity tolerances. Exact and tolerance verdicts are separate.
- **Pyflate:** dev/pyflate/rs_check.py checks output bytes against bz2.decompress, MD5, intermediate L-vector and ending bit position. Output validation is outside headline timing.

Equality is specific to the tested inputs and build. Numerical equality differs from byte identity for signed zeros; consult each checker's selected contract and output log.

A2. Matched Python/native CPU counters

The counter tables use `report/measure_native_counters.py` and the raw files in `results/vm_release_20260907/counters/`. Release CPython 3.10.12 executes the same benchmark source in explicit Python and native modes, on one fixed guest CPU. Each configuration has three runs per event pass; backend order alternates between rounds. Every run warms up before measurement. Nbody resets state afterwards and executes 64 iterations of energy, 20,000 steps and energy; pyflate executes 16 complete decompressions. Output checks follow the timer.

Counter scope: `perf stat --delay=-1 --control fifo:...` starts with counters disabled. The worker enables counting after imports, setup, warmup and garbage collection, waits for acknowledgement, then times the loop. It disables counting afterwards and waits for a second acknowledgement. Counts include the small control-boundary overhead but exclude setup and output validation. The internal elapsed timer covers the loop only. Both core and generic cache events request user mode (:u); separate event passes are separate executions.

Pass	Events and validation
Core	Cycles, instructions, branches, branch misses: retained; reported running time 100%.
Cache	Generic cache references/misses: retained; reported running time 100%.
L1 events	Not requested: an earlier capture returned invalid zero load counts. No L1 miss-rate claim is made.

The summarizer checks source SHA-256 hashes, requested backends, output SHA-256 equality across all 24 runs, nonzero denominators and counter running time. A separate dispatch check (`report/check_pyflate_backend.py`) observed zero native calls for Python and one per block for native/auto, with identical MD5 and output length. The measured pyflate extension is the refactored build; its binary hash, Rust toolchain version and source hashes are recorded in `vm_release_20260907/protocol.json`. Correctness logs identify the loaded extension path.

Reported counts are normalized per completed benchmark iteration. Each displayed value is the median of three observations, including rates calculated **within each run**. Thus a ratio of displayed median counts may differ slightly from the displayed median rate. `summary.json` also retains the individual values and minimum/maximum. Three runs support a descriptive comparison, not a formal significance claim; the rigorous 120-value timing experiments remain the headline evidence.

Interpretation: IPC is retired instructions per counted cycle, not work completed per cycle. Removing interpreter instructions can lower IPC while improving runtime substantially. Generic cache events do not represent all memory accesses or identify stall cycles, a specific cache level, or DRAM latency. Nbody's very small miss counts are particularly sensitive to boundary overhead and run-to-run variation. Pyflate's larger remaining counts cannot be assigned to BWT without an isolated phase measurement.

To regenerate the validated summaries from the preserved capture:

```
python3 report/summarize_counters.py \
  --directory results/vm_release_20260907/counters
```

For a new capture on Linux, choose a fresh output directory:

```
python3 report/measure_native_counters.py --repo /path/to/repo \
  --out /tmp/new-native-counters
```

A3. Earlier phase counters: evidence and limits

`dev/pyflate/phase_cpi.py` repeats one phase after preparing its input. The internal wall timer covers the repeated phase; `perf stat` wraps the **entire process**, including imports and input preparation. The values below reproduce `results/pyflate_phase_cpi.txt`; wall times are divided by repeat count and rounded. These are phase timing and aggregate-counter measurements, not a CPI stack decomposing execution into stall causes.

Phase	Repeats	ms/iter	IPC	Cache misses / refs
Native decode	300	3.304	1.49	1.284%
Whole inverse BWT	20	137.720	2.92	2.601%
Chase loop	30	47.820	2.41	1.831%
RLE4	100	23.836	2.89	1.623%

IPC uses instructions:u / cycles:u; the generic cache events were collected without that user-only modifier. Ratios are process-wide. Cache events do not identify which access missed, the latency it incurred, or how much useful work overlapped it. Therefore we do not infer an exact DRAM-access interval, or assign all cycles per traversal step to interpreter overhead.

The chase loop performs `end = T[end]`, reads `L[end]`, writes an output byte and runs Python loop machinery. Its setup builds `T` outside the wall timer but inside the counter interval. The `L`-vector contains 336,184 bytes; `T` is a Python list of indices, whose list slots and integer objects add memory. The final 399,360-byte output size is not the traversal table size. Whole BWT includes substantial table-construction work as well as the dependent traversal.

PMU configuration: Event passes use at most four counters and validate counts and running time. On the tested VM, fixed-period cycle sampling and software cpu-clock sampling worked where frequency-mode cycle sampling returned zero samples. Debug-build C stacks use DWARF unwinding; `py-spy` separately provides Python frames. These are observed configuration limits, not universal KVM properties. Recheck the sample-rate setting after tuning/reboot. To test a native BWT memory bottleneck, isolate its measured region and sweep working-set size; the present whole-process counters do not establish one.

A4. Full profiles and attribution

The recorded profiles are from revision 08da63e on 10 September; its benchmark sources match timing revision 2c8c754 (`results/profiles_20260910_08da63e/provenance.json`). `report/make_figures.py` preserves all recorded frame geometry, startup and harness context in each overview. The updated detail panels retain row-aligned crops with callers below the selected frame. Labels aggregate a function's inclusive share across its frames; outlines mark the widest individual occurrence. `report/fig/profile_counts.json` preserves both.

Original `nbody` uses debug-CPython C frames; optimized `nbody` and both `pyflate` figures use Python-frame `py-spy` sampling. The original SVGs remain in `results/` for zooming. Widths represent inclusive samples on a call path; parents include children and must not be added to them. Each graph has its own denominator, so widths cannot show absolute before/after speedup. C-profile weights sum event periods; Python-frame graphs contain only a few hundred samples. Both locate work rather than resolve small percentage differences. The sampled `pyflate` offload fraction is not precise enough to explain its native speedup through Amdahl's law without matched phase timing.

A5. Timing distributions and worker clustering

mean $\pm$ SD describes the observed mean and spread; it does not require symmetry or independence and is not a confidence interval. Skew affects how representative that summary is, while worker correlation affects uncertainty in the mean. The same result JSONs give:

```
suite=results/vm_canonical_20260910_2c8c754/suite
python3 report/summarize_distribution.py \
    "$suite"/{baseline,optimized,fallback,native}_nbody.json \
    "$suite"/{baseline,optimized,fallback,native}_pyflate.json
```

Shape: Every distribution except nbody’s native run and pyflate’s Python back end is right-skewed: the median sits below the mean and the maximum is far from both, which is the usual signature of occasional interference rather than a symmetric measurement error. In those two the median lies slightly above the mean, by 0.014 ms and 0.06 ms. The interquartile range is therefore the more informative spread, and it is between 0.4% and 2.0% of the median in every configuration.

Configuration	Median	IQR	p95	Max	Mean
Nbody original	228.84	3.09	246.29	266.97	231.20
Nbody optimized	142.64	0.52	145.95	151.09	143.13
Nbody Python	143.13	2.87	160.06	165.08	145.30
Nbody native	9.544	0.076	9.626	9.665	9.530
Pyflate original	1122.68	14.67	1142.92	1164.32	1123.49
Pyflate optimized	280.43	3.47	287.33	291.55	281.16
Pyflate Python	283.94	3.33	288.78	300.03	283.88
Pyflate native	169.67	2.06	173.39	186.74	170.01

Grouping: The 120 values are not 120 independent observations. They come from 40 worker processes, three values each, and values from one worker share that process’s memory layout, CPU placement and page-cache state. A one-way variance decomposition splits the total into a between-worker and a within-worker component; the between share is the intraclass correlation (ICC), and the variance of the mean is inflated by the design effect $\text{deff} = 1 + (m - 1) * \text{ICC}$, with $m = 3$ values per worker. The tool’s own output, verbatim:

clustering	SD-betw	SD-with	ICC	deff	SE naive	SE clust
baseline_nbody	7.899	0.602	0.994	2.99	0.7171	1.2396
optimized_nbody	1.519	0.405	0.934	2.87	0.1424	0.2411
fallback_nbody	3.831	3.198	0.589	2.18	0.4533	0.6691
native_nbody	0.000	0.052	0.000	1.00	0.0046	0.0046
baseline_pyflate	7.670	7.930	0.483	1.97	1.0030	1.4066
optimized_pyflate	1.574	2.619	0.265	1.53	0.2783	0.3444
fallback_pyflate	0.000	3.368	0.000	1.00	0.3053	0.3053
native_pyflate	1.011	2.065	0.193	1.39	0.2095	0.2468

SD and SE columns are in ms; ICC and deff are dimensionless. SE naive treats the 120 values as independent; SE clust adjusts for worker clustering under this model. Effective sample size is 120 / deff: 40.2 and 41.8 for nbody’s original and optimized runs, 55.1 and 120.0 for its Python and native runs, and 61.0, 78.4, 120.0 and 86.5 for pyflate’s original, optimized, Python and native runs.

Nbody’s original and optimized runs are almost entirely between-worker (ICC 0.99 and 0.93): a worker’s three values agree closely with each other and less well with another worker’s, so the effective sample size is about 40–42 rather than 120 and the standard error of the mean is roughly 1.7x what independence would give. All were pinned to guest CPU 0, so CPU placement is not what separates workers; these data do not identify what does. Nbody’s native run and pyflate’s Python back end show no detectable worker effect, and pyflate’s other runs are milder than nbody’s (ICC 0.19 to 0.48).

Interpretation: The runtime reductions are much larger than the estimated standard errors of the corresponding means. Small differences need a comparison-specific uncertainty estimate, ideally resampling worker groups; a dimensionless speedup cannot be compared directly with an SE in milliseconds. An ICC estimate of zero means no detected worker effect. The design-effect calculation assumes independent workers and a shared within-worker correlation; it does not account for all possible VM drift or provide a universal effective sample size.

A6. Flat function tables

report/summarize_profiles.py aggregates functions across recorded call sites and prints self and inclusive shares separately. For perf it reads the Children/Self columns; for py-spy it subtracts child samples from each frame to obtain self samples, then aggregates by function/file. The nbbody grouped self shares are 11.26% for list/index access and 16.25% for float handling, using the patterns printed in the report. Inline attribution and rounding can make SVG totals differ slightly from flat perf values.

`python3 report/summarize_profiles.py --top 10`

Outputs: results/profile_functions.txt, results/timing_distributions.txt and their JSON companions in report/fig/.

Text companions

The build compiles the same prose and tables into a text edition with flame graphs replaced by pointers to the PDF. It selects the installed extractor's table-preserving mode, then report/check_txt_tables.py verifies UTF-8 and that each checked row retains its values. The exporter problems that led to these checks are recorded in the repository's git log.

A7. Verification of the final code path on the course VM

Canonical pinned run (revision 2c8c754)

Both benchmarks' headlines come from this run. report/vm_refresh.py start uploaded a git archive of revision 2c8c754 to a fresh directory on the VM and launched report/vm_refresh_worker.py. The worker ran pyflate's Rust tests, Python API test and dispatch check against a fresh cargo build, then the timed stages – baseline, optimized, compare, wheel and native for both benchmarks – through tools/vm_run_all.sh, every timed run pinned to guest CPU 0. All 11 stages succeeded. Every JSON holds 120 values from 40 workers and records CPU affinity 0; the optimized, Python and native JSONs also record the requested and actual back end, and the native JSONs the extension hash, which matches the wheel built in the same run. tools/check_all.sh --require-native then passed all six checks, none skipped.

Comparison	Before (ms)	After (ms)	Speedup
Nbody original / optimized	231.20	143.13	1.62×
Nbody Python / native	145.30	9.530	15.25×
Pyflate original / optimized	1,123.49	281.16	4.00×
Pyflate Python / native	283.88	170.01	1.67×
Pyflate original / native	1,123.49	170.01	6.61×

Every native JSON identifies the binary installed from the wheel built in that run; rust/<crate>/wheels/PROVENANCE.json accompanies each preserved wheel. Earlier captures remain as cross-checks in their original directories, with their revision history in the repository's git log. This report revision reuses those measurements and introduces no new benchmark timing.

A8. Hardware evidence

Software times in this report are measured on the course VM (A7). Hardware figures are not: cycle counts come from RTL simulation against golden models, area from synthesis, and clock and power estimates from place-and-route static timing, all on the development host. The hardware comparisons in the two reports divide the first kind of number by the second. The denominators are the A7 table above and A3's 3.304 ms native decode phase.

All three modules now record **post-CTS** static timing and a default-activity power estimate in hw/<module>/docs/ppa.md: grape_pipeline 19.46 MHz, huffman_engine 39.9 MHz, mtf_cam 37.5 MHz — each from a run that met its constraint. The timing and power reports those numbers were read from are preserved in hw/<module>/synth/evidence/. None is a completed routed sign-off or a physical system benchmark.

grape_pipeline's synthesis-area table was measured before the final rewrite of its issue-selection logic. MTF integration records 158,441 RTL cycles; the W-sweep's 157,560-cycle default is a model result. The reports label both. The two pyflate modules are also simulated together: make -C hw/pyflate_accel sim runs the chain on the benchmark block, byte-exact over 336,184 bytes in 159,303 cycles. Platform DMA and host overhead, and energy savings, remain unmeasured. hw/docs/hardware_report.md maps every required

hardware item to its source file. The worked software examples are checked by `report/verify_examples.py`; defense questions and an evidence map are in `report/defense_guide.md`.

Step	Tool (version)	Why this tool
Design language	SystemVerilog, synthesizable subset	the same source passes Verilator and Yosys
Lint, main simulation	Verilator 5.051	compiled simulation is fast enough to run the whole benchmark input per test; line/toggle coverage
Cross-check simulation	Icarus Verilog 14.0	4-state: shows X-propagation after reset, which 2-state Verilator cannot
Testbench	cocotb 2.0.1, pyuvvm 4.0.1, cocotbext-axi 0.1.28	golden models are Python, so a Python testbench calls them directly as the scoreboard oracle; AXI bus models with back-pressure
Driver and unit tests	pytest 9.1.1	register-map checks and driver models without a simulator
Formal	SymbiYosys 0.68	proves control properties simulation only samples: FSM arcs, handshakes, list invariants
Synthesis, area	Yosys 0.68, sky130 high-density cells (tt, 25 °C, 1.8 V)	open synthesis onto real standard cells gives cell count and area
Process	SkyWater sky130	the fully open 130 nm PDK, reproducible without an NDA; no SRAM macros, so all storage is flip-flops
Place, clock tree, timing, power	OpenLane 2.3.10 (OpenROAD)	floorplan, placement, clock-tree synthesis, static timing and power estimate; routing not reached
Tool bundle	OSS CAD Suite 2026-08-26	one pinned archive gives identical versions on any machine

References: Python profile semantics — docs.python.org/3.10/library/profile.html. Version-specific bytecode — docs.python.org/3/library/dis.html. Repository and raw data — github.com/matanco64/HWSW_Project.